



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Оператор JOIN и теоретико-множественные операции

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 4

В этом семинаре

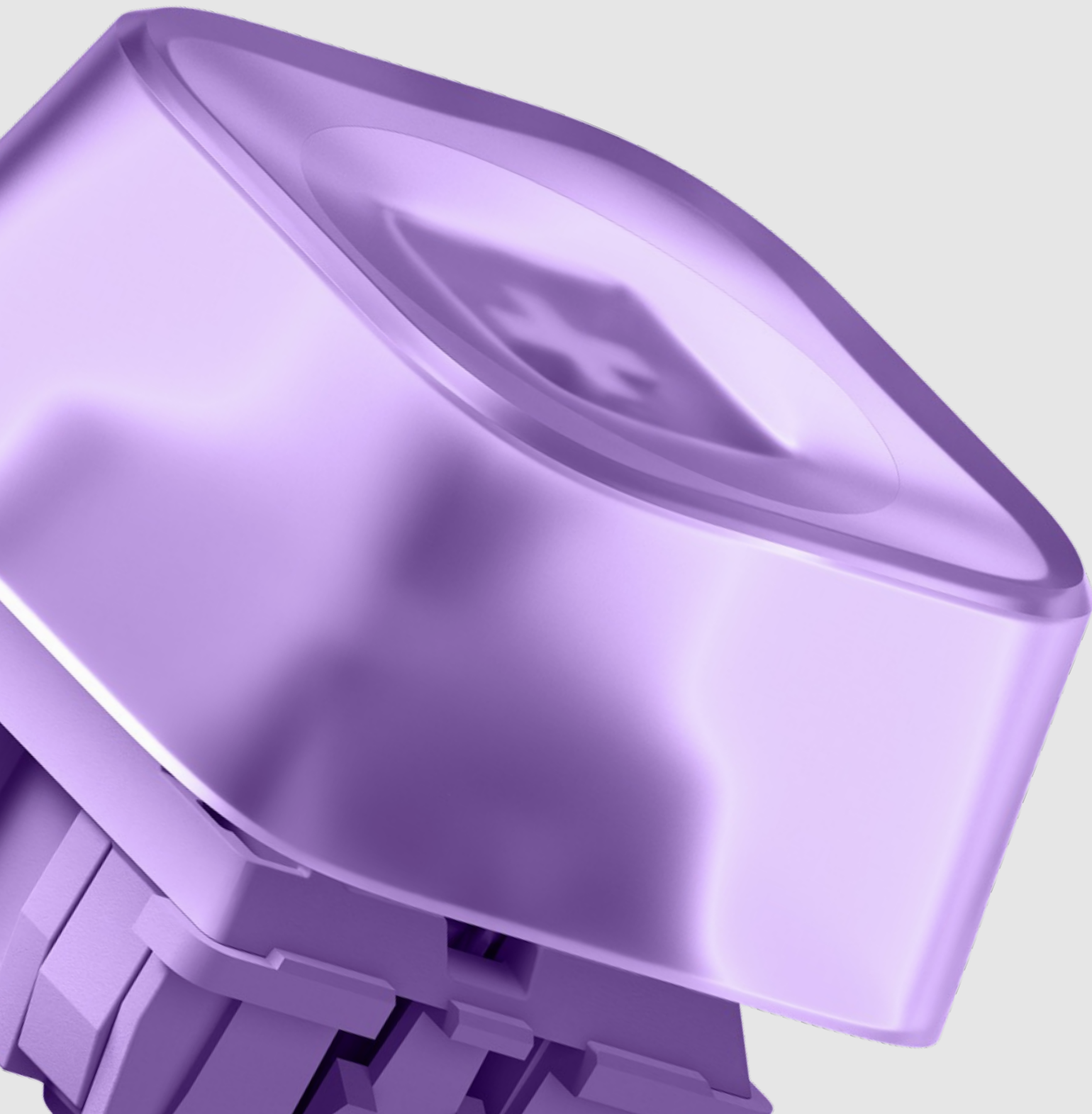
→ Выбор столбцов для вывода из разных таблиц

→ Типы соединений

→ Соединение по нескольким условиям

→ Декартово произведение таблиц

→ Соединение таблицы с собой



Соединение данных

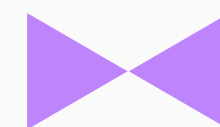
Часто нужно обращаться сразу к нескольким таблицам в запросе.

Например, добавить к платежу имя клиента из другой таблицы по его ключу (обогащение платежей данными о клиенте).



Таблица платежей

- ID платежа
- Сумма
- **ID клиента**



JOIN

Таблица клиентов

- **ID клиента**
- Город проживания
- Имя

=

Итоговая выборка

- ID платежа
- Сумма
- ID клиента
- Имя клиента

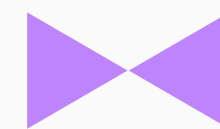
Соединение данных

Нередко необходимо использовать столбцы из разных таблиц, чтобы создать выборку, по которой можно посчитать определённую метрику.



Таблица заказов

- ID заказа
- Дата заказа
- ID клиента



JOIN

Таблица клиентов

- ID клиента
- Город проживания
- Имя

=

Итоговая выборка

- Город проживания
- Последняя дата заказа

CROSS JOIN

Декартово произведение двух таблиц

CROSS JOIN связывает каждую строку одной таблицы с другой. **CROSS JOIN** можно заменить запятой.

```
SELECT *  
FROM table1 t1  
CROSS JOIN table2 t2
```

```
SELECT *  
FROM table1 t1, table2 t2
```

CROSS JOIN

ПРИМЕР

Users

user_id	name
1	Андрей
2	Артём

Cross Join

Professions

user_id	profession	salary
1	Аналитик	100 000
2	Дизайнер	80 000
3	Преподаватель	50 000

```
SELECT *  
FROM users u, professions p
```

user_id	name	user_id	profession	salary
1	Андрей	1	Аналитик	100 000
1	Андрей	2	Дизайнер	80 000
1	Андрей	3	Преподаватель	50 000
2	Артём	1	Аналитик	100 000
2	Артём	2	Дизайнер	80 000
2	Артём	3	Преподаватель	50 000

CROSS JOIN

Декартово произведение двух таблиц

CROSS JOIN полезен, когда нужно создать все возможные комбинации данных из двух или более таблиц.

Например, при генерации списка всех возможных комбинаций размеров и цветов одежды в интернет-магазине.

```
SELECT *  
FROM table1 t1  
      CROSS JOIN table2 t2
```

CROSS JOIN

ПРИМЕР

Users

user_id	name
1	Андрей
2	Артём

Cross Join

Professions

user_id	profession	salary
1	Аналитик	100 000
2	Дизайнер	80 000
3	Преподаватель	50 000

```
SELECT *  
FROM users u, professions p
```

user_id	name	user_id	profession	salary
1	Андрей	1	Аналитик	100 000
1	Андрей	2	Дизайнер	80 000
1	Андрей	3	Преподаватель	50 000
2	Артём	1	Аналитик	100 000
2	Артём	2	Дизайнер	80 000
2	Артём	3	Преподаватель	50 000

CROSS JOIN

ПРИМЕР

Users

user_id	name
1	Андрей
2	Артём

cross join + u.user_id = p.user_id

Professions

user_id	profession	salary
1	Аналитик	100 000
2	Дизайнер	80 000
3	Преподаватель	50 000

```
SELECT *
FROM users u, professions p
WHERE u.user_id = p.user_id
```

user_id	name	user_id	profession	salary
1	Андрей	1	Аналитик	100 000
2	Артём	2	Дизайнер	80 000

По условию в **WHERE** СУБД поймёт, что тут не нужно делать **CROSS JOIN**.

Виды соединений

Внутренние

INNER JOIN (JOIN)



Внешние

LEFT / RIGHT / FULL [OUTER] JOIN



Виды соединений

Внутренние

INNER JOIN (JOIN)



Результат запроса с использованием внутреннего соединения — строки, которые совпали по условию соединения в каждой из таблиц.

Внешние

LEFT / RIGHT / FULL [OUTER] JOIN



Соединение данных

Предложение **JOIN** записывается
после **FROM**.

Далее, после **ON**, указывается
условие соединения.

INNER JOIN (JOIN) — это **CROSS
JOIN** с фильтрами, указанными
после **ON**.

```
SELECT столбец1, столбец ...  
FROM таблица1 as т1  
    JOIN таблица2 as т2  
        ON условие_соединения
```

Условия соединения

01

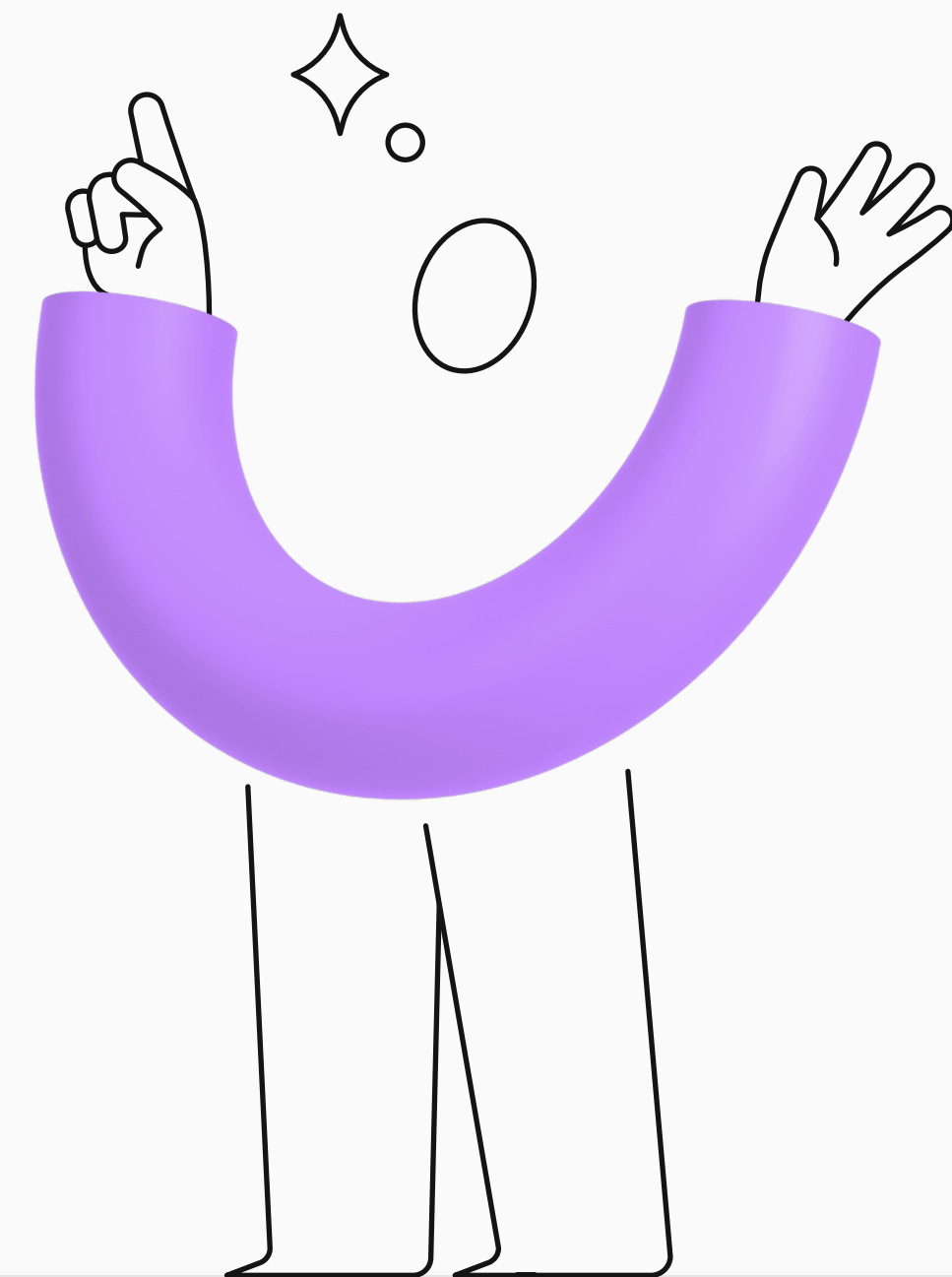
Соединение по равенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
    ON т1.столбец_соединения =  
    т2.столбец_соединения
```

Соединение по равенству

ПРИМЕР

Реализовать `INNER JOIN` для `table1` и `table2` по полю `user_id`.



Соединение по равенству

ПРИМЕР

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

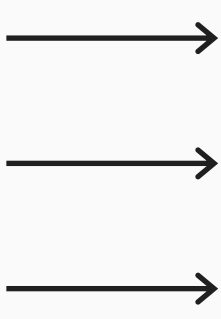


Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

Соединение по равенству

ПРИМЕР

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

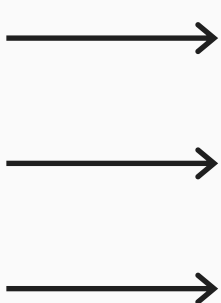


Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

```
SELECT t1.*, t2.profession
FROM table t1
INNER JOIN table2 t2
ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель

Условия соединения

01

Соединение по равенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
    ON т1.столбец_соединения =  
    т2.столбец_соединения
```

02

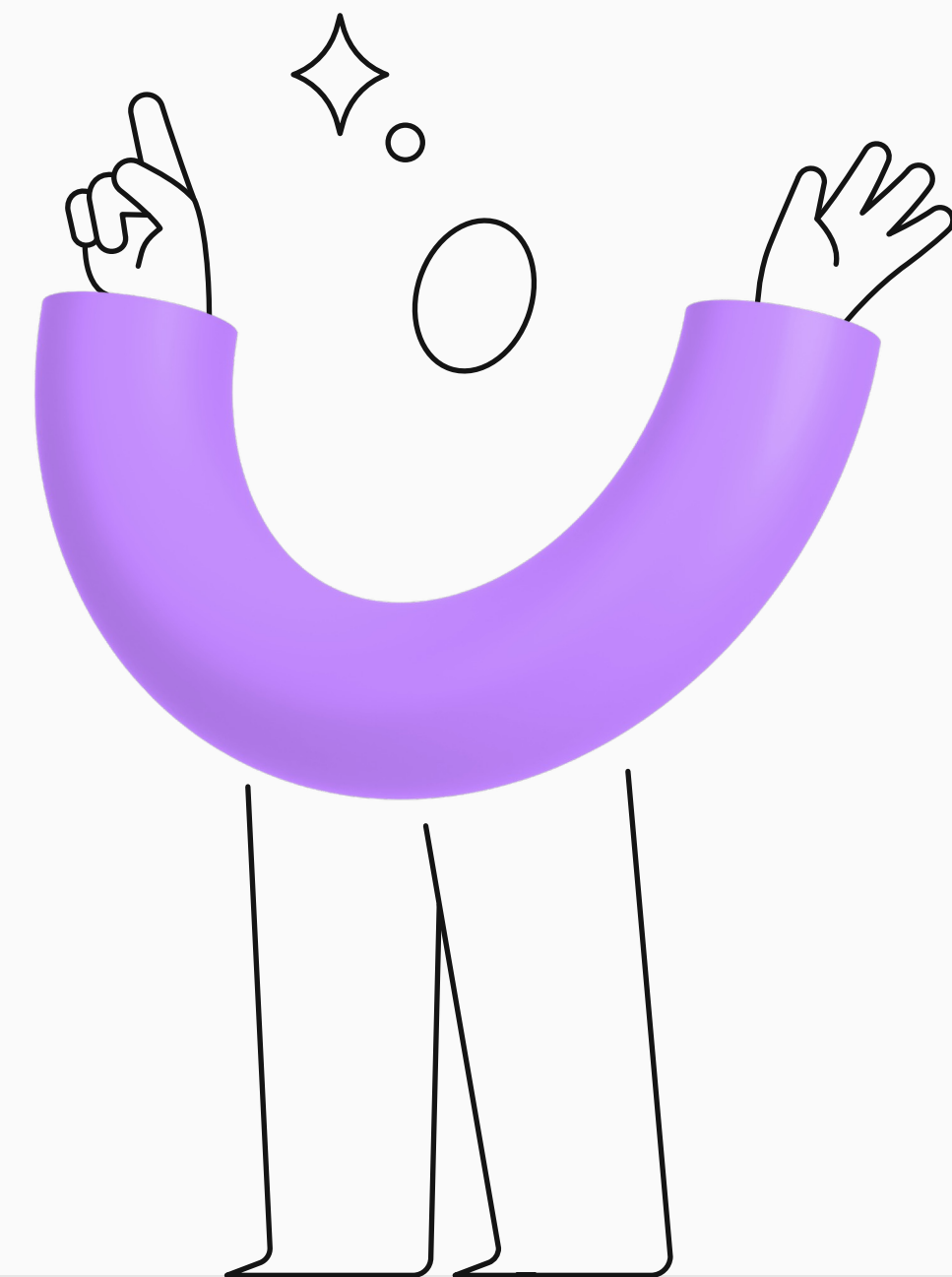
Соединение по неравенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
    ON т1.столбец_соединения >  
    т2.столбец_соединения
```

Соединение по неравенству

ПРИМЕР

Реализовать `INNER JOIN` для `table1` и `table2`, чтобы определить, какие вакансии подходят каждому клиенту.



Соединение по неравенству

ПРИМЕР

Table 1

user	salary_need
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

Table 2

profession	max_salary
Аналитик	250 000
Дизайнер	90 000
Преподаватель	60 000
Таксист	30 000

Соединение по неравенству

ПРИМЕР

Table 1

user	salary_need
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

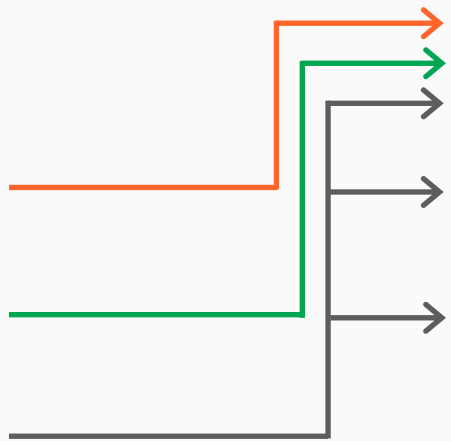


Table 2

profession	max_salary
Аналитик	250 000
Дизайнер	90 000
Преподаватель	60 000
Таксист	30 000

Соединение по неравенству

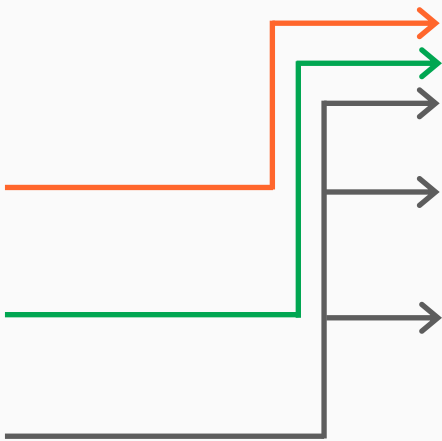
ПРИМЕР

Table 1

user	salary_need
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

Table 2

profession	max_salary
Аналитик	250 000
Дизайнер	90 000
Преподаватель	60 000
Таксист	30 000



```
SELECT *
FROM table1 as t1
JOIN table2 as t2
ON t1.salary_need <= t2.max_salary
```

user_id	salary_need	profession	max_salary
Артём	100 000	Аналитик	250 000
Мария	150 000	Аналитик	250 000
Георгий	50 000	Аналитик	250 000
Георгий	50 000	Дизайнер	90 000
Георгий	50 000	Преподаватель	60 000

Соединение по неравенству

ПРИМЕР

Сравнение по неравенству — очень затратная операция для соединения. Чаще всего при использовании неравенства используется самый долгий алгоритм соединения, поэтому по возможности не стоит использовать неравенства для соединения таблиц при написании запросов.



INNER JOIN

ЗАДАЧА 1

Вывести из таблицы клиентов (**shop_customers**) уникальный список с ID клиента, именем и фамилией тех, кто делал покупки (**shop_orders**) за с 1 января 2024 г. по 31 марта 2024 г.

INNER JOIN

ЗАДАЧА 1

Вывести из таблицы клиентов (**shop_customers**) уникальный список с ID клиента, именем и фамилией тех, кто делал покупки (**shop_orders**) за с 1 января 2024 г. по 31 марта 2024 г.

```
SELECT DISTINCT c.customer_id
, c.first_name
, c.last_name
FROM shop_customers c
INNER JOIN shop_orders o
ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-03-31'
```


Условия соединения

01

Соединение по равенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON т1.столбец_соединения =  
    т2.столбец_соединения
```

02

Соединение по неравенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON т1.столбец_соединения >  
    т2.столбец_соединения
```

03

Соединение по всегда
верному условию

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON 1=1
```

Соединение по всегда верному условию

Таблица 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

```
SELECT *
FROM таблица1 т1
JOIN таблица2 т2
ON 1=1
```

Таблица 2

user_id	name
1	Аналитик
2	Дизайнер
3	Преподаватель
4	Таксист

Соединение по всегда верному условию

Таблица 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

```
SELECT *  
FROM таблица1 т1  
JOIN таблица2 т2  
ON 1=1
```

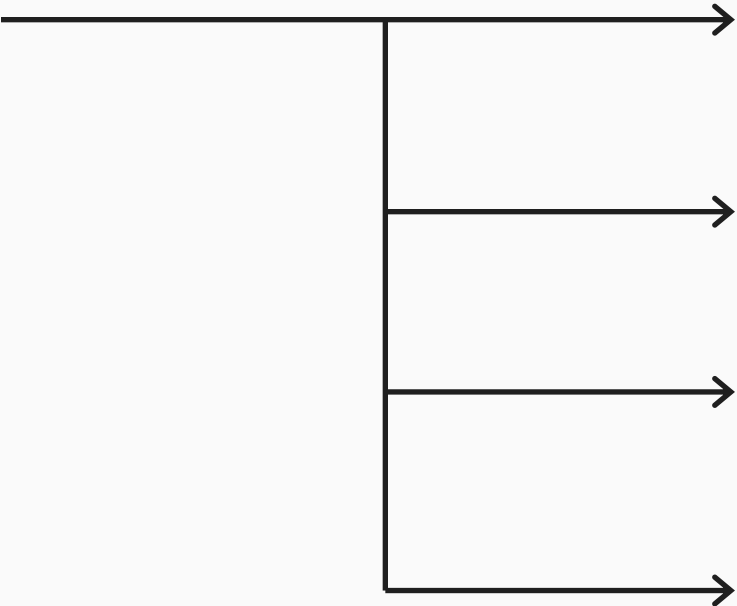


Таблица 2

user_id	name
1	Аналитик
2	Дизайнер
3	Преподаватель
4	Таксист

Аналогично для каждой из строк «таблица 1».

Соединение по всегда ложному условию

Table 1

user_id	name
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

```
SELECT *  
FROM таблица1 т1  
JOIN таблица2 т2  
ON 1=0
```



Table 2

user_id	name
Андрей	300 000
Артём	100 000
Мария	150 000
Георгий	50 000

Какой будет результат?

Условия соединения

01

Соединение по равенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON т1.столбец_соединения =  
    т2.столбец_соединения
```

02

Соединение по неравенству

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON т1.столбец_соединения >  
    т2.столбец_соединения
```

03

Соединение по всегда
верному условию

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON 1=1
```

04

Соединение по нескольким
условиям

```
SELECT *  
FROM таблица1 т1  
    JOIN таблица2 т2  
        ON т1.столбец_соединения1 =  
    т2.столбец_соединения2  
        AND т1.столбец_соединения1 =  
    т2.столбец_соединения2 ...
```

Соединение по нескольким условиям

ПРИМЕР

Можно осуществлять соединения таблиц по комбинации условий. Тут так же, как и в случае с **WHERE**, используем для этого **AND/OR**.

Задача: соединить таблицы из примера по `user_id` и условию, что нам нужна информация только об аналитиках.



Соединение по нескольким условиям

ПРИМЕР

```
SELECT t1.*, t2.profession
FROM table1 t1
  INNER JOIN table2 t2
    ON t1.user_id = t2.user_id
   AND t2.profession = 'Аналитик'
```

Соединение по нескольким условиям

ПРИМЕР

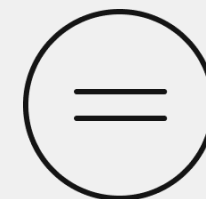
```
SELECT t1.*, t2.profession
FROM table1 t1
     INNER JOIN table2 t2
         ON t1.user_id = t2.user_id
         AND t2.profession = 'Аналитик'
```

```
SELECT t1.*, t2.profession
FROM table1 t1
     INNER JOIN table2 t2
         ON t1.user_id = t2.user_id
WHERE t2.profession = 'Аналитик'
```


Соединение по нескольким условиям

ПРИМЕР

```
SELECT t1.*, t2.profession
FROM table1 t1
     INNER JOIN table2 t2
           ON t1.user_id = t2.user_id
           AND t2.profession = 'Аналитик'
```

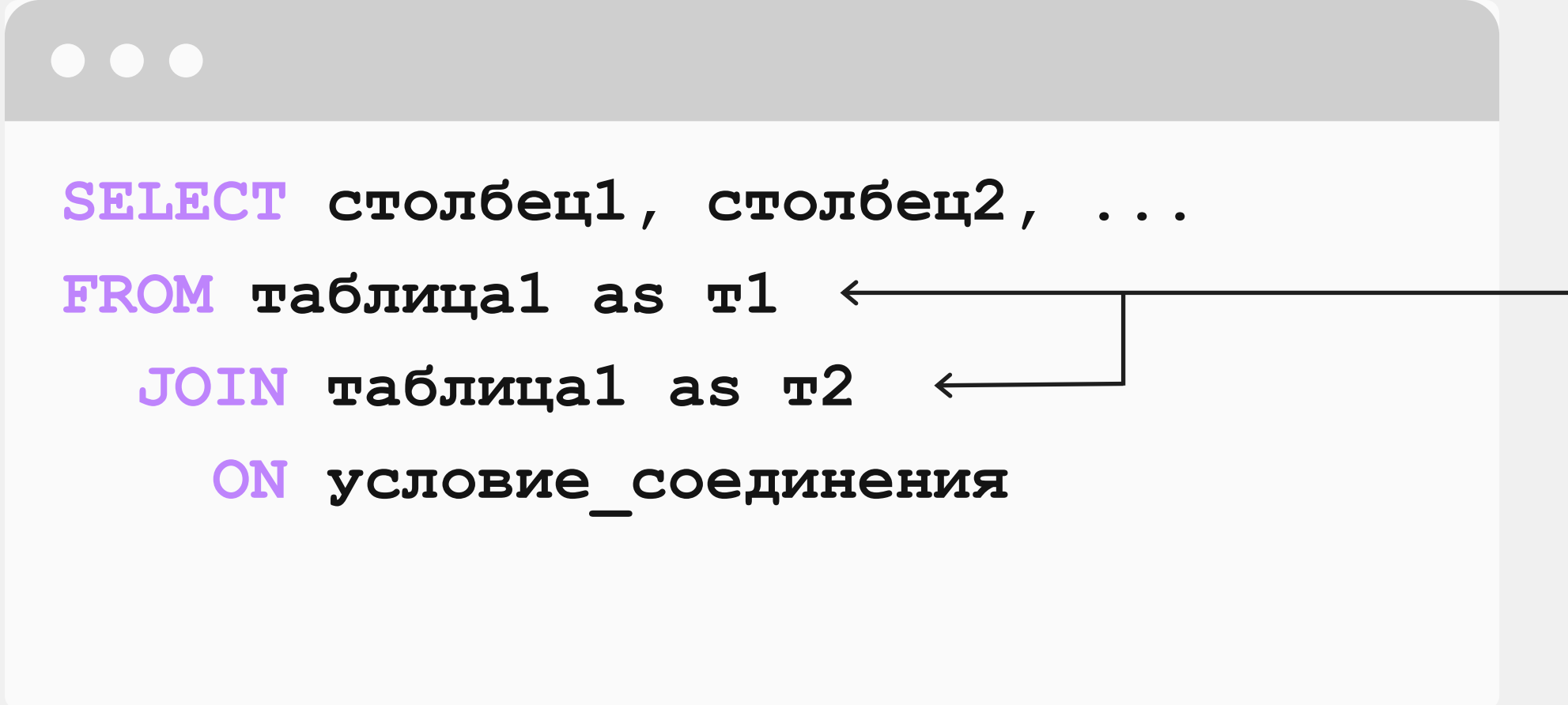


```
SELECT t1.*, t2.profession
FROM table1 t1
     INNER JOIN table2 t2
           ON t1.user_id = t2.user_id
WHERE t2.profession = 'Аналитик'
```

В **INNER JOIN** условия из **WHERE** можно переносить в условие соединения.

Результат запроса останется **идентичным**.

Использование одной и той же таблицы в join



```
SELECT столбец1, столбец2, ...  
FROM таблица1 as т1  
JOIN таблица1 as т2  
ON условие_соединения
```

Иногда называется SELF JOIN.

Мы можем использовать одну и ту же таблицу в запросе неограниченное число раз с разными алиасами.

Часто применяется для анализа иерархических данных, поиска связей между записями внутри таблицы.

Использование одной и той же таблицы в join

Вывести рядом с именем сотрудника, имя его руководителя.
Информация о руководителе хранится в той же таблице.

emp

emp_id	name	manager_id
1	Андрей	3
2	Артём	3
3	Мария	4
4	Георгий	Null

```
SELECT
  u.name as emp_name,
  m.name as manager_name
FROM emp as e
JOIN emp as m
  ON e.manager_id = m.user_id
```

result

emp_name	manager_name
Андрей	Мария
Артём	Мария
Мария	Георгий

Здесь таблица emp используется
для подстановки имени менеджера
по его идентификатору.

Внешнее соединение данных

Используя **INNER JOIN** в результате запроса, можем потерять строки, которым не нашлось пары в одной из таблиц. Можно ли как-то этого избежать?

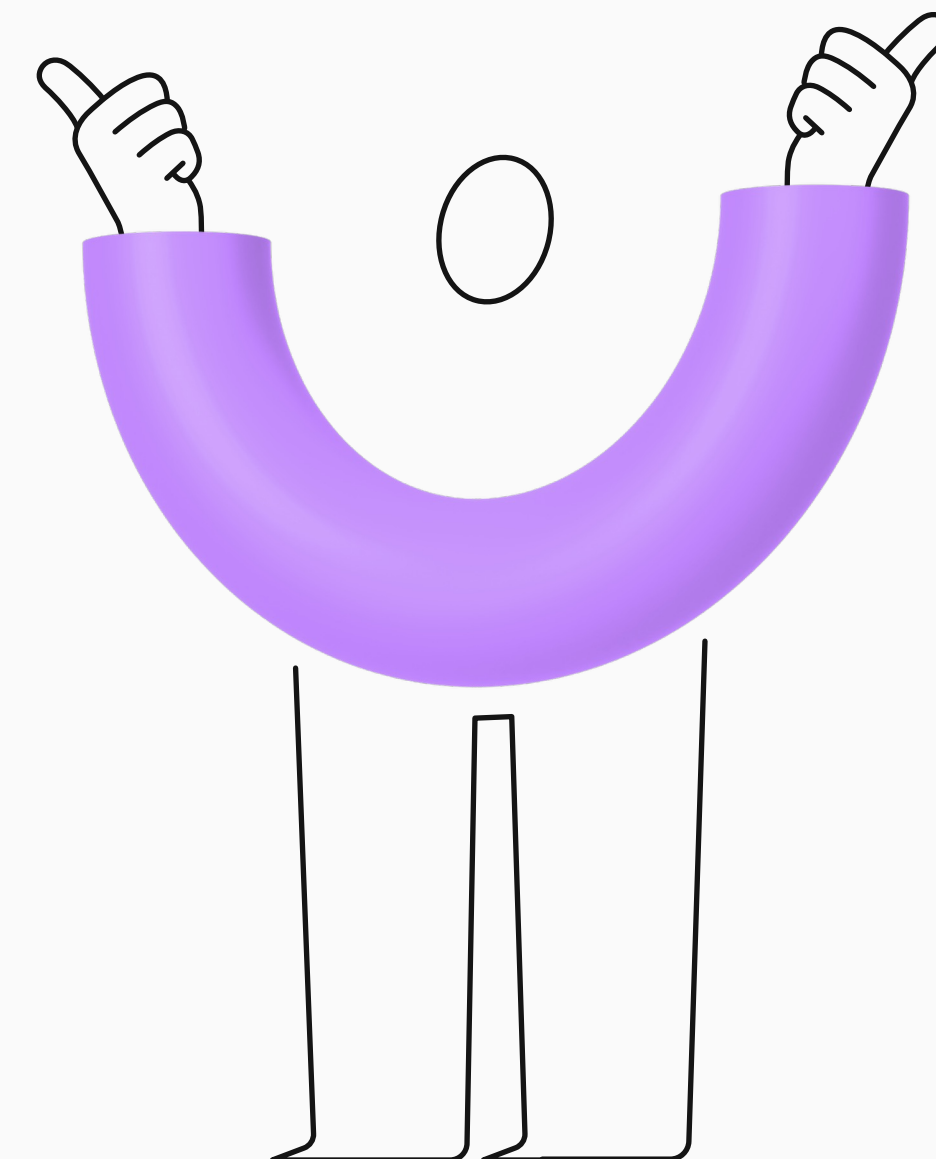


Внешнее соединение данных

Используя **INNER JOIN** в результате запроса, можем потерять строки, которым не нашлось пары в одной из таблиц. Можно ли как-то этого избежать?



Да! Для этого существуют **внешние соединения**.



Виды соединений

Внутренние

INNER JOIN (JOIN)



Внешние

LEFT / RIGHT / FULL [OUTER] JOIN



Виды соединений

```
SELECT *  
FROM table1 t1  
    LEFT JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Для строк в таблице 1,
которым не нашлось пары
в таблице 2, значения во всех
столбцах из таблицы 2 будут
заполнены **NULL**.

Виды соединений

```
SELECT *  
FROM table1 t1  
    LEFT JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Для строк в таблице 1,
которым не нашлось пары
в таблице 2, значения во всех
столбцах из таблицы 2 будут
заполнены **NULL**.

```
SELECT *  
FROM table1 t1  
    RIGHT JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Для строк в таблице 2,
которым не нашлось пары
в таблице 1, значения во всех
столбцах из таблицы 1 будут
заполнены **NULL**.

Виды соединений

```
SELECT *  
FROM table1 t1  
    LEFT JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Для строк в таблице 1,
которым не нашлось пары
в таблице 2, значения во всех
столбцах из таблицы 2 будут
заполнены **NULL**.

```
SELECT *  
FROM table1 t1  
    RIGHT JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Для строк в таблице 2,
которым не нашлось пары
в таблице 1, значения во всех
столбцах из таблицы 1 будут
заполнены **NULL**.

```
SELECT *  
FROM table1 t1  
    FULL JOIN table2 t2  
        ON t1.поле_соединения =  
t2.поле_соединения
```

Все строки обеих таблиц.

Типы внешних соединений

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

Типы внешних соединений

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

```
SELECT t1.*, t2.profession
FROM table1 t1
LEFT JOIN table2 t2
ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
4	Георгий	NULL

Типы внешних соединений

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

```
SELECT t1.*, t2.profession
FROM table1 t1
  RIGHT JOIN table2 t2
    ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
NULL	NULL	Таксист

Типы внешних соединений

Table 1

user_id	name
1	Андрей
2	Артём
3	Мария
4	Георгий

Table 2

user_id	profession
1	Аналитик
2	Дизайнер
3	Преподаватель
5	Таксист

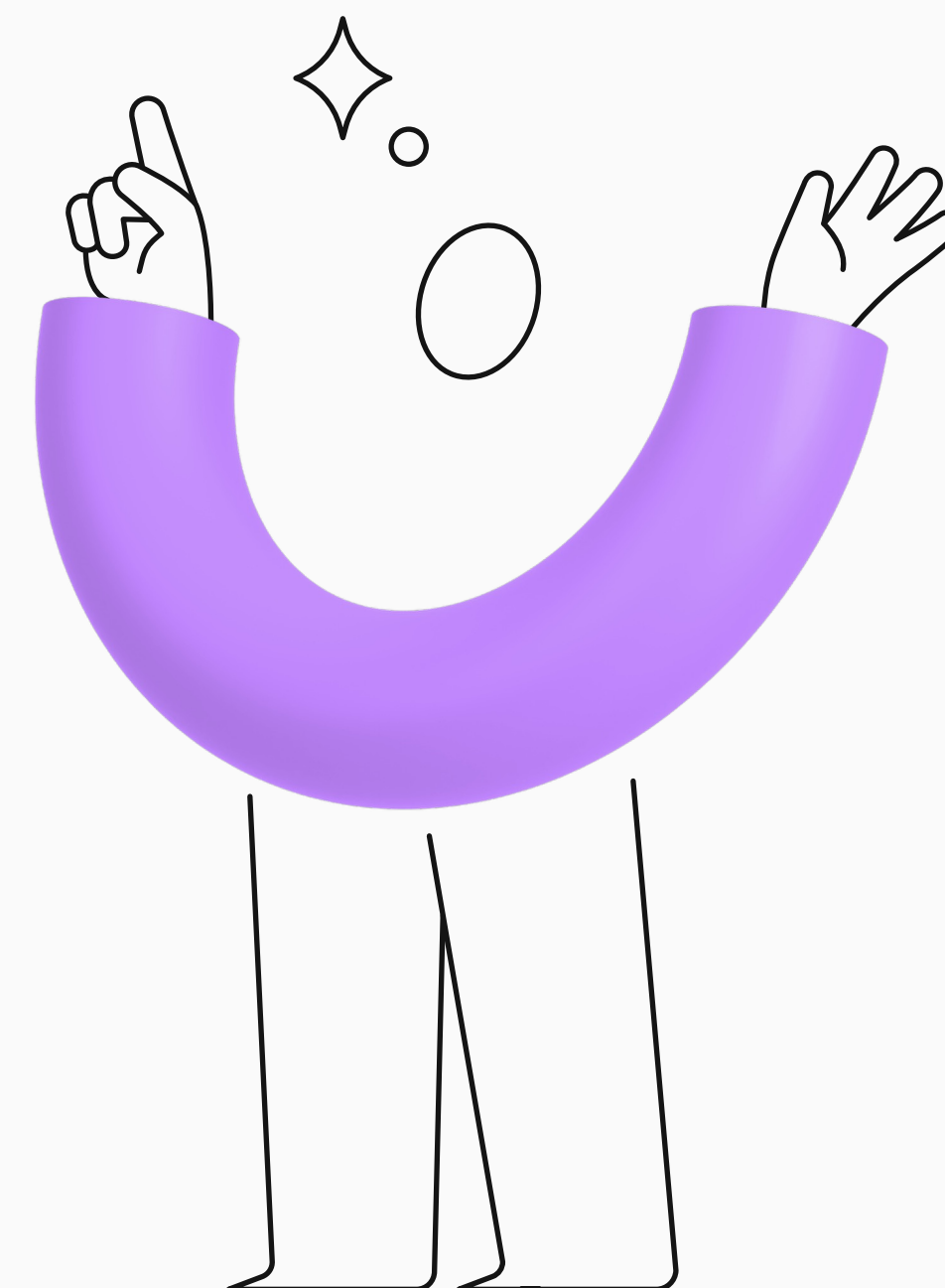
```
SELECT t1.*, t2.profession
FROM table1 t1
FULL JOIN table2 t2
ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
4	Георгий	NULL
NULL	NULL	Таксист

Типы внешних соединений

ПРИМЕР

Вывести из Table 1 ID клиентов, чью профессию мы не знаем (такой нет в нашей таблице профессий пользователей).



Типы внешних соединений

ПРИМЕР

Шаг 1. Нам нужно определить профессию для всех пользователей из таблицы 1.

Значит, делаем **LEFT JOIN**.

```
SELECT t1.*, t2.profession
FROM table1 t1
LEFT JOIN table2 t2
ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
4	Георгий	NULL

Типы внешних соединений

ПРИМЕР

Шаг 2. Определяем пользователей, по которым не нашлось информации о профессии.

```
SELECT t1.*, t2.profession
FROM table1 t1
LEFT JOIN table2 t2
ON t1.user_id = t2.user_id
```

user_id	name	profession
1	Андрей	Аналитик
2	Артём	Дизайнер
3	Мария	Преподаватель
4	Георгий	NULL

Типы внешних соединений

ПРИМЕР

Шаг 3. Реализуем условие **WHERE** в запросе.

```
SELECT t1.user_id
FROM table1 as t1
LEFT JOIN table2 as t2
ON t1.user_id = t2.user_id
WHERE t2.profession is NULL
```

user_id
4

Типы внешних соединений

ПРИМЕР

Шаг 3. Реализуем условие **WHERE** в запросе.

```
SELECT t1.user_id
FROM table1 as t1
LEFT JOIN table2 as t2
ON t1.used_id = t2.user_id
WHERE t2.profession is NULL
```

user_id
4

Перенесём условия в **JOIN**.

≠

```
SELECT t1.user_id
FROM table1 as t1
LEFT JOIN table2 as t2
ON t1.used_id = t2.user_id
AND t2.profession is NULL
```

user_id
1
2
3
4

LEFT/RIGHT JOIN

ЗАДАЧА 3,4

3

Вывести из таблицы заказов (shop_orders) ID заказа, дату заказа и статус заказа, а также метод оплаты (взять в таблице shop_payments).

4

Сколько заказов с пустым значением payment_method в платеже?

LEFT/RIGHT JOIN

ЗАДАЧА 2,3

2

Вывести из таблицы заказов (shop_orders) ID заказа, дату заказа и статус заказа, а также метод оплаты (взять в таблице shop_payments).

```
SELECT o.order_id
      , o.order_date
      , o.order_status
      , p.payment_method
FROM shop_orders o
LEFT JOIN shop_payments p
      ON o.order_id = p.order_id
```

3

Сколько заказов с пустым значением payment_method в платеже?

```
SELECT COUNT(distinct o.order_id)
FROM shop_orders o
LEFT JOIN shop_payments p
      ON o.order_id = p.order_id
where p.payment_method IS NULL
```

LEFT/RIGHT JOIN

ЗАДАЧА 4

Вывести из таблицы клиентов (**shop_customers**) ID клиента, имя, фамилию, также рассчитать число оставленных отзывов, если такие имеются. Если клиент не оставлял отзывы, то **он всё равно должен попасть в выборку**.

LEFT/RIGHT JOIN

ЗАДАЧА 4

Вывести из таблицы клиентов (`shop_customers`) ID клиента, имя, фамилию, также рассчитать число оставленных отзывов, если такие имеются. Если клиент не оставлял отзывы, то он всё равно должен попасть в выборку.

```
SELECT c.customer_id
      , c.first_name
      , c.last_name
      , COUNT(distinct review_id) AS cnt
FROM shop_customers c
LEFT JOIN shop_reviews r
      ON c.customer_id = r.customer_id
GROUP BY c.customer_id
      , c.first_name
      , c.last_name
```


Множественные JOIN

- Работаем с тремя и более таблицами.
- Реальные бизнес-задачи могут требовать написания запросов с 3 и более JOIN.

```
SELECT *  
FROM table1 t1  
    INNER JOIN table2 t2  
        ON t1.user_id = t2.user_id  
    LEFT JOIN table3 t3  
        ON t2.order_id = t3.user_id  
    LEFT JOIN table4 t4  
        ON t3.product_id = t4.product_id  
...
```

Множественные JOIN

ЗАДАЧА 5

Для каждого клиента из таблицы **shop_customers** посчитать число уникальных продуктов, которые он покупал. В ответ вывести ID клиента и число уникальных товаров.

Если клиент не покупал ни одного продукта, то он всё равно должен попасть в выборку с **0** в столбце числа уникальных продуктов.

Множественные JOIN

ЗАДАЧА 5

Для каждого клиента из таблицы `shop_customers` посчитать число уникальных продуктов, которые он покупал. В ответ вывести ID клиента и число уникальных товаров.

Если клиент не покупал ни одного продукта, то он всё равно должен попасть в выборку с 0 в столбце числа уникальных продуктов.

```
SELECT
    c.customer_id
    ,count(distinct oi.product_id) as cnt
FROM shop_customers c
LEFT JOIN shop_orders o
    ON c.customer_id = o.customer_id
LEFT JOIN shop_orderitems oi
    ON oi.order_id = o.order_id
GROUP BY 1
```

Заключение

Типы операторов JOIN

01

INNER JOIN

Оставляет записи обеих таблиц при наличии совпадения по условию JOIN.

02

LEFT (RIGHT) JOIN

Оставляет все записи первой (второй) таблицы, подтягивая к ним только те значения второй (первой) таблицы, которые совпадают по условию JOIN.

03

FULL JOIN

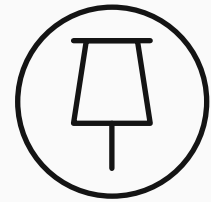
Оставляет записи обеих таблиц, даже при отсутствии совпадения по условию JOIN.

04

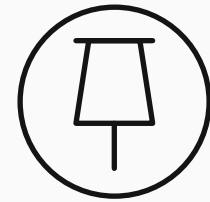
CROSS JOIN

Декартово произведение таблиц без дополнительных условий.

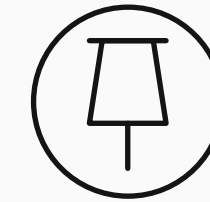
Операции над множествами



Операции над множествами
UNION, EXCEPT, INTERSECT



Работа с уникальными
строками и дублирующими
строками при использовании
операций над множествам



Сортировка, фильтрация
и группировка строк
в операциях над множествами

Когда нужны операции над множествами

→ Объединить две таблицы с одинаковой структурой в одну.

→ Найти отличия в двух наборах данных.

→ Найти совпадающие строки в наборах данных.

Для этих ситуаций **JOIN** не подходит.



Почему такую операцию мы не можем сделать через JOIN?

Тип	Название	Население
Город	Москва	13 000 000
Город	Санкт-Петербург	5 000 000

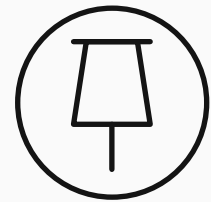
Тип	Название	Население
Страна	Россия	130 000 000
Страна	Бразилия	140 000 000



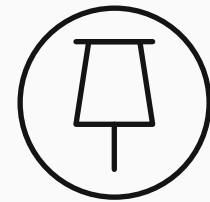
Тип	Название	Население
Город	Москва	13 000 000
Город	Санкт-Петербург	5 000 000
Страна	Россия	130 000 000
Страна	Бразилия	140 000 000

Операции над множествами

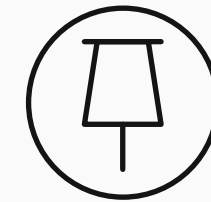
Что делает каждая операция?



Объединение **UNION**



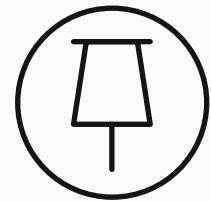
Пересечение **INTERSECT**



Разность **EXCEPT**

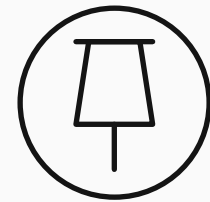
Операции над множествами

Что делает каждая операция?



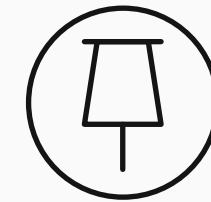
Объединение **UNION**

Объединяет и возвращает строки множеств.



Пересечение **INTERSECT**

Находит одинаковые строки в обоих множествах и возвращает их.



Разность **EXCEPT**

Возвращает из первого множества строки, которых нет во втором.

Требования к таблицам (запросам)

В операциях над множествами

- Одинаковое число атрибутов.
- Совпадающий тип данных в соответствующих атрибутах.
- Можно использовать как сложные запросы, так и полные таблицы.
- Название столбцов берется из первого запроса.

Синтаксис операции

Применение операций над множествами к таблицам

- Объединение таблиц X и Y — **UNION** (**ALL**) :
- Разность таблиц X и Y — **EXCEPT** (**ALL**) :
- Пересечение X и Y — **INTERSECT** (**ALL**) :

В SQL эти операции будут возвращать уникальные строки.

Можно отменить исключение дубликатов, используя слово **ALL**. Например, **UNION ALL**.



```
SELECT ...  
FROM ...  
ОПЕРАЦИЯ  
SELECT ...  
FROM ...  
ОПЕРАЦИЯ  
SELECT ...  
FROM ...  
ОПЕРАЦИЯ  
.....
```

ЗАДАЧА 6

Формирование полного списка клиентов для рассылки

Задача: объединить `customer_id` клиентов с заказами и отзывами.

Ответ должен содержать колонки:

`customer_id` — идентификатор клиентов из таблиц заказов (`shop_orders`) и отзывов (`shop_reviews`).

Используйте таблицы `shop_orders` и `shop_reviews`.

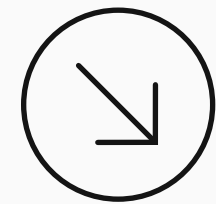
ЗАДАЧА 6

Формирование полного списка клиентов для рассылки

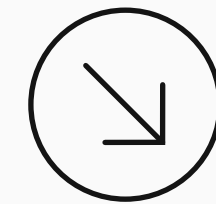
Задача: объединить `customer_id` клиентов с заказами и отзывами.

```
SELECT customer_id
FROM shop_orders
UNION
SELECT customer_id
FROM shop_reviews;
```


Что делает **ALL**?



В **UNION ALL** отключает неявную дедубликацию.



В **INTERSECT ALL** и **EXCEPT ALL** начинает учитывать количество вхождений дубликатов.

ЗАДАЧА 7

Поиск клиентов с учётом их активности

Задача: объединить `customer_id` из таблиц заказов (`shop_orders`) и ОТЗЫВОВ (`shop_reviews`), оставляя дубликаты.

Формат вывода:

`customer_id` — идентификатор клиентов из таблиц заказов и отзывов.

ЗАДАЧА 7

Поиск клиентов с учётом их активности

Задача: объединить `customer_id` из таблиц заказов (`shop_orders`) и ОТЗЫВОВ (`shop_reviews`), оставляя дубликаты.

Формат вывода:

`customer_id` — идентификатор клиентов из таблиц заказов и отзывов.

```
--Запрос про заказы
SELECT customer_id
FROM shop_orders

-- Без ALL не будет работать,
так как нам нужны дубли
UNION ALL

-- Запрос про отзывы
SELECT customer_id
FROM shop_reviews
```


Самая частая операция — UNION

→ **UNION** требует больше ресурсов.

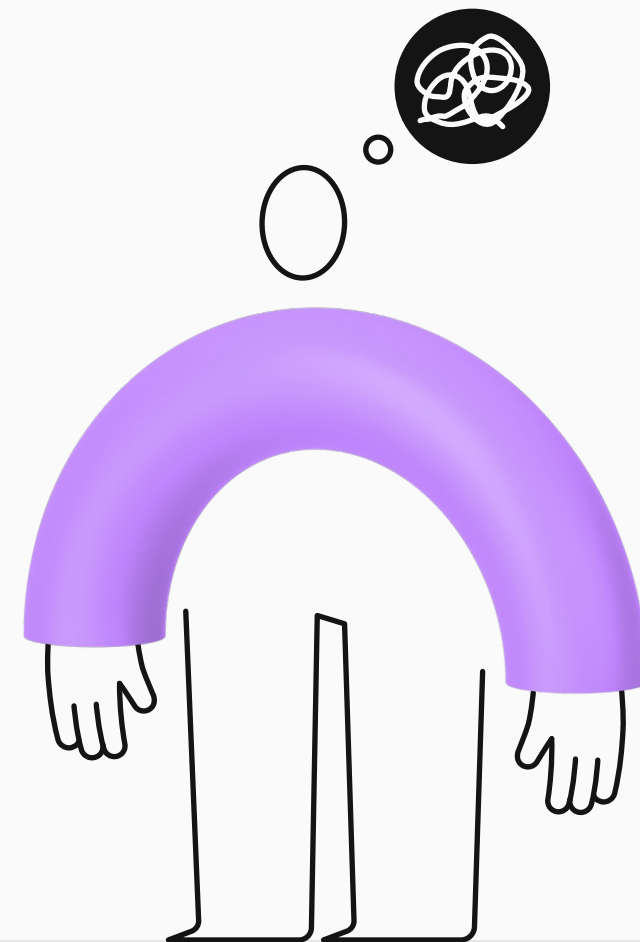
→ **UNION ALL** требует меньше ресурсов.

UNION использует неявную агрегацию для удаления дублей и тратит больше ресурса.



Применение сортировки вместе с UNION

Допускается применять сортировку с **UNION**: можно её использовать внутри запроса или снаружи?



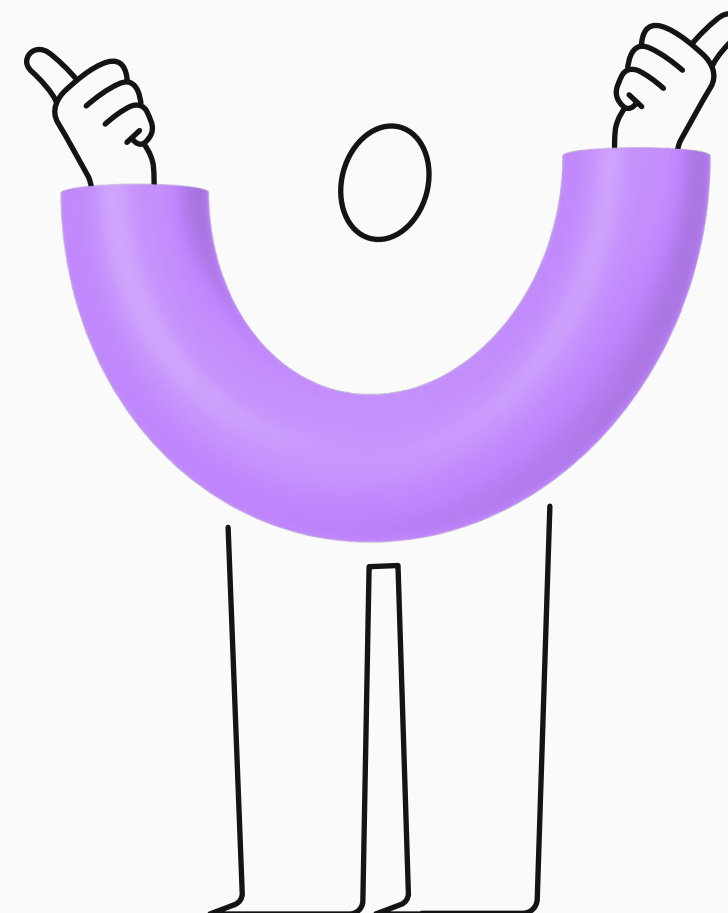
Применение сортировки вместе с UNION

Допускается применять сортировку внутри **UNION** и снаружи. Это влияет на результат запроса.

Когда это полезно?

Для сортировки всего результата запроса.

Для того, чтобы оставить только часть данных одного из множеств.



Применение сортировки вместе с UNION

Допускается применять сортировку внутри **UNION** и снаружи. Это влияет на результат запроса.

```
(SELECT ...  
ORDER BY ...)
```

UNION

```
(SELECT ...  
ORDER BY ...)
```

```
SELECT ...
```

UNION

```
SELECT ...  
ORDER BY
```

```
(SELECT ...  
ORDER BY ...)
```

UNION

```
(SELECT ...  
ORDER BY ...)
```

```
ORDER BY
```

ЗАДАЧА 8

Выбор наиболее активных клиентов по недавней истории заказов и отзывов

Бизнес-цель: получить несколько наиболее активных клиентов по дате для быстрой кампании по данным покупок и отзывов.

Задача: объединить топ-3 `customer_ID` из недавних заказов и топ-3 из отзывов.

Формат вывода:

- `customer_id` — идентификатор клиентов из таблиц заказов (`shop_orders`) и отзывов (`shop_reviews`);
- `date` — дата отзыва (`created_at`) из таблицы (`shop_reviews`) или заказа (`order_date`) из таблицы заказов (`shop_orders`).

Отсортировать общий результат по `date`.

ЗАДАЧА 8

Выбор наиболее активных клиентов по недавней истории заказов и отзывов

```
(SELECT customer_id, order_date AS date
FROM shop_orders
ORDER BY order_date DESC
LIMIT 3)
UNION
(SELECT customer_id, created_at
FROM shop_reviews
ORDER BY created_at DESC
LIMIT 3)
order by date
```

Будет применяться только к первому запросу.
Если убрать скобки, то будет ошибка.

Будет применяться ко всему запросу.
Используется вне скобок.

Фильтрация строк в UNION

Допускается применять фильтрацию строк только внутри **UNION**.
Использовать скобки необязательно.

```
(SELECT ...  
WHERE ...)  
UNION  
(SELECT ...  
WHERE ...)
```

```
(SELECT ... )  
UNION  
(SELECT ...)  
WHERE
```

```
SELECT ...  
WHERE ...  
UNION  
SELECT ...  
WHERE ...
```

Какой из представленных шаблонов с ошибкой?

Фильтрация строк в UNION

Допускается применять фильтрацию строк только внутри **UNION**.
Использовать скобки необязательно.

```
(SELECT ...  
WHERE ...)  
UNION  
(SELECT ...  
WHERE ...)
```

```
(SELECT ... )  
UNION  
(SELECT ...)  
WHERE
```

```
SELECT ...  
WHERE ...  
UNION  
SELECT ...  
WHERE ...
```

Какой из представленных шаблонов с ошибкой?

Фильтрация строк в UNION

Чтобы фильтровать результат операции над множествами, необходимо их поместить в подзапрос.

```
(SELECT ...  
WHERE ...)  
UNION  
(SELECT ...  
WHERE ...)
```

```
SELECT *  
FROM (  
    (SELECT ... )  
    UNION  
    (SELECT ...)  
) AS T  
WHERE ...
```

```
SELECT ...  
WHERE ...  
UNION  
SELECT ...  
WHERE ...
```

Группировка строк в UNION

Допускается применять группировку строк только внутри **UNION**.
Использовать скобки необязательно.

```
(SELECT ...  
GROUP BY ...)  
UNION  
(SELECT ...  
GROUP BY ...)
```

```
(SELECT ... )  
UNION  
(SELECT ...)  
GROUP BY
```

```
SELECT ...  
GROUP BY ...  
UNION  
SELECT ...  
GROUP BY ...
```


Группировка строк в UNION

Допускается применять группировку строк только внутри **UNION**.
Использовать скобки необязательно.

```
(SELECT ...  
GROUP BY ...)  
UNION  
(SELECT ...  
GROUP BY ...)
```

```
SELECT *  
FROM (  
    (SELECT ... )  
    UNION  
    (SELECT ...)  
) t  
GROUP BY
```

```
SELECT ...  
GROUP BY ...  
UNION  
SELECT ...  
GROUP BY ...
```

ЗАДАЧА 9

Выявление 25 самых активных пользователей

Бизнес-цель: найти 25 самых активных пользователей (по отзывам и покупкам).

Задача: объединить `customer_id` из заказов и отзывов, сохраняя дубликаты. Затем посчитать общее количество взаимодействий пользователя. (Эта задача является продолжением предыдущей задачи. Рекомендуется использовать код из предыдущего решения.)

Формат вывода:

- `customer_id` — идентификатор клиентов из таблиц заказов (`shop_orders`) и отзывов (`shop_reviews`);
- `interaction_count` — количество взаимодействий (как сумма числа отзывов и заказов).

ЗАДАЧА 9

Выявление 25 самых активных пользователей

Бизнес-цель: найти 25 самых активных пользователей (по отзывам и покупкам).

```
select customer_id, count(*) as interaction_count
from
  (--Запрос про заказы
  SELECT customer_id
  FROM shop_orders
  -- Без ALL не будет работать, так как нам нужны дубли
  UNION ALL
  -- Запрос про отзывы
  SELECT customer_id
  FROM shop_reviews) t
group by customer_id
order by number_of_itteraction DESC
Limit 25;
```

Если убрать **all**, то **interaction_count** будет всегда 1 из-за дедубликации.

Чтобы применить группировку, ко всему результату, пришлось поместить запрос с **UNION ALL** в подзапрос.

EXCEPT, INTERSECT

Фильтрация, сортировка, группировка

Во всех остальных операциях над множествами работают те же принципы, что и в **UNION**.

ЗАДАЧА 10

Формирование полного списка клиентов с указанием типа активности

Бизнес-цель: найти клиентов с недавней активностью. Нужно определить последние взаимодействия каждого клиента по отзывам (**shop_reviews**) и покупкам (**shop_orders**).

Задача: написать запрос, который сможет найти последнюю известную активность каждого клиента в отзывах и заказах.

Итоговый результат нужно отсортировать результат по идентификатору клиента и типу активности и ограничить, чтобы он содержал не более 30 строк.

Формат вывода:

- **customer_id** — ключ клиента;
- **activity_dt** — дата последней активности. **Order_date** для заказов или **created_at** для отзывов;
- **activity_type** — тип активности, варианты значений которого приведены в описании задачи в воркбуке.

ЗАДАЧА 10

Формирование полного списка клиентов с указанием типа активности

Бизнес-цель: найти клиентов с недавней активностью.

```
(select distinct on(customer_id) customer_id,
order_date as activity_dt
, 'покупка' as activity_type
FROM databases_and_sql.shop_orders
order by customer_id, order_date desc)
UNION ALL
(select distinct on(customer_id) customer_id,
created_at, 'отзыв'
FROM databases_and_sql.shop_reviews
order by customer_id, created_at desc)
order by activity_dt desc
limit 300
```

В данном случае можно использовать **ALL**, чтобы не делать лишнюю дедубликацию, за счёт наличия **distinct on**, а также констант, это значит, что дублей быть не может.

Сортировка применяется ко второму. За счёт **distinct on** остаётся только по одному экземпляру по **customer_id**.

Сортировка применяется ко всему запросу.

ЗАДАЧА 11

Сравнение результатов двух запросов

Задача: с помощью SQL нужно показать, что приведённые запросы возвращают одинаковый результат.

Эти запросы были написаны для решения следующей задачи: **найти страны и города, у которых совпадает название.**

```
-- Запрос 1
select name
from w_city c
intersect
select name
from w_country c2;

-- Запрос 2
select c.name
from w_city c
    join w_country c2
        on c."name" = c2."name"
```

ЗАДАЧА 11

С помощью SQL нужно показать, что приведённые запросы возвращают одинаковый результат.

Найти страны и города, у которых совпадает название.

ШАГ 1

Сначала докажем, что запрос 1 не отличается от запроса 2.

-- Запрос 1

```
select name  
from w_city c  
intersect  
select name
```

```
from w_country c2;
```

EXCEPT

Если результат — пустое множество, то в запросе 1 нет отличных от запроса 2 строк.

-- Запрос 2

```
select c.name  
from w_city c  
      join w_country c2  
        on c."name" = c2."name"
```


ЗАДАЧА 11

С помощью SQL нужно показать, что приведённые запросы возвращают одинаковый результат.

Найти страны и города, у которых совпадает название.

ШАГ 2

Теперь докажем, что запрос 2 не отличается от запроса 1.

```
-- Запрос 2
select c.name
from w_city c
      join w_country c2
        on c."name" = c2."name"
```

EXCEPT

```
-- Запрос 1
select name
from w_city c
intersect
select name
from w_country c2;
```

Если результат —
пустое множество,
то в запросе 2 нет
отличных от запроса
1 строк.

ЗАДАЧА 11

С помощью SQL нужно показать, что приведённые запросы возвращают одинаковый результат.

Найти страны и города, у которых совпадает название.

ШАГ 3

Теперь надо объединить запросы с шага 1 и шага 2.

```
-- Запрос 1
select name
from w_city c
intersect
select name
from w_country c2;
```

EXCEPT

```
-- Запрос 2
select c.name
from w_city c
join w_country c2
on c."name" = c2."name»
```

UNION ALL

```
-- Запрос 2
select c.name
from w_city c
join w_country c2
on c."name" = c2."name"
```

EXCEPT

```
-- Запрос 1
select name
from w_city c
intersect
select name
from w_country c2;
```

Если результат — пустое множество, то запрос 1 не отличается от запроса 2. И запрос 2 не отличается от запроса 1.

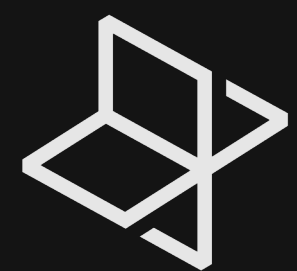
Итоги занятия

→ Операции над множествами нужны, когда нужно объединить две и более таблицы или сравнить их.

→ Существуют следующие операции над множествами:

- **UNION** — объединение;
- **EXCEPT** — разность;
- **INTERSECT** — пересечение.

→ В некоторых задачах необходимо использовать уникальные значения, а в других необходимо сохранить дубликаты через **ALL**. Также за счёт отсутствия скрытой дедупликации при использовании **ALL** такие операции работают быстрее. Поэтому часто можно встретить **UNION ALL** в крупных БД.



**ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ**

Вопросы